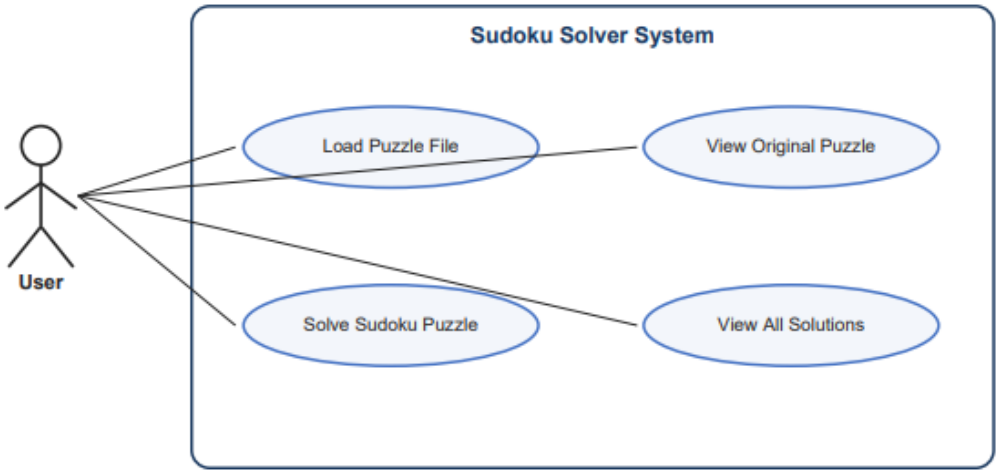
a. The main use case is Solve Sudoku Puzzle. The user runs the program to load each puzzle, see the original puzzle, and view every valid solution found. The supporting use cases shown are Load Puzzle File, View Original Puzzle, and View All Solutions. The User is an external actor because they start the program and receive its output. The system boundary contains the functionality provided by the Sudoku solver. The diagram uses association lines to show which actor interacts with the system.



Solid lines show the User's association with system use cases.

b. The use case allows the user to run the Sudoku solver on five puzzle files. The program uses recursive depth-first search with backtracking to find valid solutions. It displays each file name, the original puzzle, and the solutions found, or “No solution found” if a puzzle has no solution.

c.

1. The user launches Assignment4.exe.
2. The program prepares the five input filenames: puzzle1.txt through puzzle5.txt.
3. For each file, the program attempts to open and read the puzzle.
4. The program checks that the puzzle input contains 81 cells represented by digits 1–9 or blank cells marked with an underscore.
5. The program checks the starting puzzle for conflicts in each row, column, and 3×3 block, if that validation is implemented in the submitted code.
6. The program displays the filename and original puzzle.
7. The solver finds an empty cell and tries a candidate digit that does not conflict with the current row, column, or 3×3 block.
8. The solver continues recursively. If a choice leads to a dead end, it removes that choice and tries another candidate (backtracking).
9. Whenever no empty cells remain and the grid is valid, the program displays the solution and continues searching for additional solutions.

10. After the search ends, the program reports the number of solutions or displays “No solution found.” It then processes the remaining files and finishes.